

Conception et Implémentation du Système d'Information pour l'Épicerie "Au Valaisan"

Projet Bases de Données 2025-2026

ARDAN Fatima, DIAKITE Ousmane, MENGOSSE Adrien, NAMATY
Yasser, OUDRHIRI Safwane

GRENOBLE-INP ENSIMAG

1^{er} décembre 2025

Plan de la Présentation

- 1 Introduction
- 2 Analyse Fonctionnelle
- 3 Conception du Modèle
- 4 Fonctionnalités Critiques
- 5 Démonstration
- 6 Bilan

1. Contexte et Objectifs du Projet

- **Le Client** : Épicerie locale « **Au Valaisan** », spécialisée en vente de **vrac** et **produits locaux**.
- **La Mission** : Modernisation complète du Système d'Information (SI).
- **Objectifs Techniques** :
 - ▶ Concevoir un modèle de données robuste (**BCNF**).
 - ▶ Implanter sur le SGBD **Oracle**.
 - ▶ Développer un démonstrateur **Java** / **JDBC** pour les transactions.
- **Livrable Clé** : Implémenter et valider **3 fonctionnalités critiques** du métier.

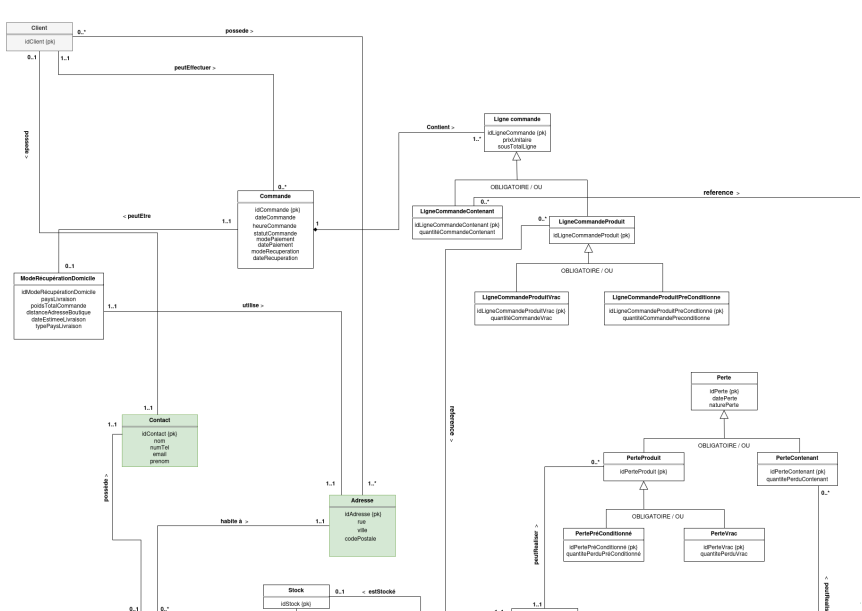
2. Analyse : Les Points Clés du Modèle Métier

- **Produits / Producteurs** : Modélisation de la relation ($N : M$) et des informations de contact/géolocalisation du producteur.
- **Conditionnements** : Distinction essentielle entre **Vrac** (poids) et **Pré-conditionné** (unité/sachet) pour les prix.
- **Lots & Dates Limite** : Entité Lot pour la traçabilité avec DLC ou DLUO (contrainte d'exclusion).
- **Saisonnalité** : Gérée par l'entité Disponibilité (dates de début/fin).
- **Commandes** : Modélisation du cycle de vie (statut) pour gérer les actions autorisées (annulation, paiement).
- **Clients & Droit à l'oubli** : Séparation des données pour permettre l'anonymisation tout en conservant l'historique de commande.

3. Contraintes Métier Majeures

- **Péréemption à J+7 (CT)** : Déclenchement automatique d'alertes et de réduction de prix.
- **Gestion du Stock (CT)** : Stratégie **multi-lots** avec obligation de sortir les lots les plus anciens ou les plus périssables en premier (FIFO, FEFO).
- **Concurrence SQL (CT)** : Le stock est **toujours disponible** même si en préparation → nécessité de verrouillage pour l'intégrité.
- **Types de Date Limite (CV)** : Strictement DLUO ou DLC (contrainte CHECK ou trigger).
- **Annulation selon Statut (CT)** : Possible pour le client en statut "En Préparation", pour l'épicerie jusqu'à "Expédié".
- **Intégrité des Données** : Assurer la cohérence des stocks, des prix et des montants des lignes de commande.

4. Modèle Entité-Association (Vue Globale)



- **Organisation Produit–Producteur** : relation structurée (1, N) permettant la gestion des stocks multi-producteurs.
- **Conditionnement** : super-entité distinguant **Vrac** (poids) et **Préconditionné** (unités), essentielle pour le calcul du prix et la gestion de stock.
- **Lots et Péremption** : chaque lot possède une date limite et assure la traçabilité ; gestion cohérente entre **date limite**, **quantité disponible** et **alertes**.

5. Points Essentiels du Modèle E/A — Métier

- **Structure du Stock** : le stock relie Producteur, Produit et Lots ; permet une gestion fine des quantités selon les arrivages.
- **Contenant Réutilisable** : séparé du Produit, car il peut être vendu indépendamment ou lié à la vente en vrac.
- **Lignes de Commande** : distinction entre ligne produit et ligne contenant (héritage ou association faible).
- **Mode de Récupération de la Commande** : livraison ou retrait en boutique, influant sur les frais et le traitement.
- **Droit à l'Oubli** : séparation entre identité du client et historique, permettant anonymisation sans perte de cohérence.

6. Traduction Relationnelle et Normalisation

- **Tables Principales** : PRODUCTEUR, PRODUIT, CLIENT, COMMANDE, LOT, CONDITIONNEMENT (env. 20 tables au total).
- **PK / FK** : Clés primaires et clés étrangères définies pour garantir l'intégrité référentielle.
- **Formes Normales** : Schéma relationnel atteint la **3ème Forme Normale (3NF)**, voire la **FNBC** pour les tables les plus importantes (justification par les DF).
- **Contraintes Non Traduisibles** :
 - ▶ Contraintes d'exclusion (DLC ou DLUO) : Implémentées via CHECK ou Trigger.
 - ▶ Contraintes dynamiques ($\text{Stock} \leq \text{Capacité}$) : Gérées par Triggers ou Transactions.

7. Fonctionnalité 1 : Passage d'une Commande (T1)

- ❶ **Vérification** : Stock Suffisant et Saisonnalité du produit.
- ❷ **Calculs Initiaux** : Sous-Total des lignes de commande, Frais de livraison estimés.
- ❸ **Création Commande** : Insertion de la Commande et des Lignes de Commande.
- ❹ **Statut Initial** : Statut = "*EnPreparation*".

Intégrité et Concurrency

- **Transaction Atomique (COMMIT/ROLLBACK)** : Garantit que toute la commande est insérée ou rien.
- **Gestion Concurrency** : SELECT FOR UPDATE sur les lignes de stock lors de la vérification finale pour éviter la survente.

8. Fonctionnalité 1 : Passage d'une Commande (T1)

Fonctionnalité 1 : Passage d'une commande par un client

Cette transaction doit :

- Vérifier la disponibilité de chaque produit commandé (stock suffisant, respect de la saisonnalité)
- Calculer le montant total de la commande (produits + éventuels frais de livraison)
- Gérer le stock selon la stratégie choisie (voir section 2.5)
- Créer la commande avec le statut approprié
- Gérer les cas d'erreur (stock insuffisant, produit hors saison)

```
public String passerCommande(String idClient,
    // On new ArrayList pour stocker les commandes qui sont p2
)
{
    // Validation: livraison domicile requiert paiement en ligne
    if ("Domicile".equalsIgnoreCase(modeRecuperation) && !"En ligne".equalsIgnoreCase(modePaiement)) {
        throw new IllegalArgumentException("Le paiement en ligne est obligatoire pour une livraison à domicile");
    }
    if ("Domicile".equalsIgnoreCase(modeRecuperation) && (idAdresseDomicile == null || distanceLivraison == null || typePayLivraison == null)) {
        throw new IllegalArgumentException("Les informations de livraison (adresse, distance, type pays) sont obligatoires pour une livraison à domicile");
    }

    Connection conn = DataSourceProvider.getValidConnection();
    conn.setAutoCommit(false);
    conn.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);

    try {
        String idCommande = generateId("@PREFIX@C@");
        //LocalDate dateDateLivraison = LocalDate.now();
        afficherColonnes(conn, idCommande);
        afficherTable(conn, "BIB0808@LigneCommandeProduitVrac");

        // 1) Créer la commande (statut En préparation)
        String sqlCommande = "INSERT INTO Commande (idCommande, dateCommande, heureCommande, statutCommande, modePaiement, modeRecuperation, idClient) " +
            "VALUES (?, TRUNC(SYSDATE), SYSTIMESTAMP, 'En preparation', ?, ?, ?)";
        System.out.println("(DEBUG) SQL Commande: " + sqlCommande);
        try (PreparedStatement ps = conn.prepareStatement(sqlCommande)) {
            ps.setString(1, idCommande);
            ps.setString(2, modePaiement);
            ps.setString(3, modeRecuperation);
            ps.setString(4, idClient);
            ps.executeUpdate();
        }
    }
}
```

9. Fonctionnalité 1 : Passage d'une Commande (T1)

```
public String passerCommande(String idClient,
    String idProduit = it.getIdProduit();
    String idProducteur = it.getIdProducteur();
    String typeCord = it.getTypeCordTissagement();
    double quantite = it.getQuantite();
    // 4) verifier la disponibilité
    boolean dispo = produitDAO.estDisponible(idProduit, idProducteur, new java.sql.Date(System.currentTimeMillis()));
    if (!dispo) {
        // recuperer la prochaine disponibilité
        java.sql.Date prochainDispo = produitDAO.getProchainDisponible(idProduit, idProducteur);
        String message = "Produit hors saison: " + idProduit;
        if (prochainDispo != null) {
            message += "Prochaine disponibilité : " + prochainDispo.toLocalDate();
        } else {
            message += "Aucune période de disponibilité future prévue.";
        }
        throw new SQLException(message);
    }
    // 5) verifier stock
    String idStock = stockDAO.getIdStock(idProduit, idProducteur, conn);
    if (idStock == null) throw new SQLException("Stock introuvable pour le produit: " + idProduit);
    StockService stockService = new StockService(idStock);
    ArrayList<String> lots_prix = stockService.stockSuffisantProduit(idProduit, idProducteur, quantite, dateEstimeeLivraison, typeCord, conn);
    if (lots_prix.isEmpty()) throw new SQLException("Stock insuffisant pour le produit: " + idProduit);
    // 6) gets unit price
    Float prix_u = produitDAO.getPrixVenteClient(idProduit, idProducteur);
    if (prix_u == null) throw new SQLException("Prix introuvable pour le produit: " + idProduit);
    double sousTotal = prix_u * quantite;
```

```
    }
}

// 5) Si Domicile avec paiement En ligne, enregistrer datePaiement immédiatement
if ("Domicile".equalsIgnoreCase(modeRecuperation) && "En ligne".equalsIgnoreCase(modePaiement)) {
    commandeDAO.enregistrerDatePaiement(idCommande);
}

conn.commit();
if ("Boutique".equalsIgnoreCase(modeRecuperation)){
    ClotureCommande clotureCommande = new ClotureCommande();
    clotureCommande.cloturerCommande(idCommande);
    conn.commit();
    return idCommande;
}

catch (SQLException e) {
    try { conn.rollback(); } catch (SQLException ignore) {}
    throw e;
} finally {
    try {
        if ("Boutique".equalsIgnoreCase(modeRecuperation)){
            conn.setAutoCommit(true); } catch (SQLException ignore) {}
    }
}
```

11. Fonctionnalité 2 : Gestion des Alertes de Péréemption

Objectif : assurer le suivi des lots sensibles et maintenir la cohérence du stock.

- **Détection des lots périssables** : identification des lots dont la DLC/DLUO est dans les **7 jours**.
- **Intégration des pertes (casse / vol)** : récupération via l'entité *Perte*, fusionnées avec les alertes de péréemption dans une **vue unifiée**.
- **Interface graphique enrichie** : distinction visuelle entre **PEREMPTION** et **PERTE**, statuts dynamiques : *En attente*, *Réduction appliquée*, *Perte détectée*.
- **Application d'une réduction de 30%** : effectuée au niveau du *Conditionnement* du produit.
- **Sécurisation de la transaction** : utilisation de **SELECT FOR UPDATE** pour garantir :
 - ▶ qu'une réduction ne soit appliquée qu'une seule fois ;
 - ▶ la cohérence des prix en cas d'accès concurrent.

12. Fonctionnalité 3 : Clôture d'une Commande (T3)

- ➊ **Mise à jour Statut** : Passage au statut "*Prte*".
- ➋ **Décrément Stock** : *Sortie effective du stock*. Décrémentation des Lots selon la politique (Ex : FEFO).
- ➌ **Paieement** : Si Mode Paiement = Boutique, l'enregistrement du paiement est finalisé à ce stade.
- ➍ **Finalisation Livraison** : Si Mode Récupération = Domicile, calcul des frais finaux et enregistrement de la date effective de livraison/retrait.
- ➎ **Blocage** : L'annulation client est définitivement bloquée.

13. Démonstration (Scénario Guidé)

- **Jeu de Données** : Préparation d'un jeu de données réaliste (avec un Lot à J+7 et un Lot presque vide).
- **Scénario** :
 - ➊ Affichage de l'**Alerte de péremption** (Lot A).
 - ➋ Exécution de la **Réduction de prix** (-30%) sur le Lot A.
 - ➌ **Passage d'une commande** (T1) sur un produit critique (gestion de la concurrence).
 - ➍ Enregistrement d'une Perte sur un Contenant.
 - ➎ **Clôture** de la commande (T3) → Vérification de la décrémentation du stock.

14. Bilan du Projet et Difficultés

- Schéma relationnel **cohérent** (BCNF/3NF) malgré les spécificités.
- **Robustesse** : Gestion validée de la **concurrency** sur le stock.
- **Organisation** : Utilisation du modèle **DAO** en Java et de **GitLab**.

14. Bilan du Projet et Difficultés

- Schéma relationnel **cohérent** (BCNF/3NF) malgré les spécificités.
- **Robustesse** : Gestion validée de la **concurrency** sur le stock.
- **Organisation** : Utilisation du modèle **DAO** en Java et de **GitLab**.
- **Modélisation** : Temps considérable passé en équipe, puis **erreurs détectées en codage** qui ont nécessité une lourde révision (rollback).
- **Technique** : Implantation exigeante dans l'environnement **ORACLE** (contraintes, triggers).
- **Droit à l'Oubli** : Exige une attention particulière sur la conservation de l'historique et l'anonymisation.

15. Conclusion et Remerciements

- L'importance cruciale de la **qualité de la modélisation E/A** avant tout codage.
- Maîtriser les **transactions complexes** et les enjeux de la **concurrency SQL**.
- Le travail collaboratif structuré sur un projet SGBD de A à Z.

Merci de votre attention. **Questions ?**